Übungsblatt 7: CPU-Architekturen

Mikrocomputertechnik 3 · Zeitpläne, Registerversionen und Vorhersage auf Papier

Ziel und Arbeitsweise

Die sechs Pflichtaufgaben sind auf Papier lösbar. Sie zählen Takte in festgelegten Modellen, verfolgen alte und neue Werte und unterscheiden Berechnung von endgültiger Bestätigung.

Bearbeitungszeit: etwa 60–75 Minuten selbstständig. **Orientierungspunkte:** 32. Im Unterricht: Aufgabe 1 (10 min), Aufgaben 2–3 (15 min), Aufgabe 4 (15 min), Aufgaben 5–6 (15 min). Besprechungen gehören zu den Vorlesungsphasen.

CPU bedeutet Central Processing Unit, ISA Instruction Set Architecture, IPC Instructions Per Cycle und CPI Cycles Per Instruction. IPC und CPI zählen hier bestätigte ISA-Befehle und dieselben Kerntakte. Issue ist der Start, Retire oder Commit die endgültige Bestätigung. Geben Sie Rechenweg und Voraussetzungen an. Alle Zeiten sind Lehrannahmen.

Aufgabe 1: Pipeline und Laufzeitfenster (6 Punkte)

M1 besitzt fünf Stufen zu je einem Takt und startet höchstens einen neuen Befehl pro Takt. Es gibt zunächst keine Abhängigkeit oder Wartezeit. Das Fenster beginnt beim ersten Start und endet am letzten WB, Write Back, Register-Rückschreiben.

1. Fünf unabhängige Befehle werden ausgeführt. Bestimmen Sie Fensterlänge, CPI und IPC. Warum ist die mittlere Rate hier noch nicht ein Befehl je Takt? (2 P)
2. Nun liest der zweite Befehl das Ergebnis des ersten. Kein Forwarding, direkte Ergebnisweiterleitung, ist vorhanden. Der Produzent rechnet in EX, Execute, Takt drei und schreibt in WB fünf. Schreiben vor Registerlesen im selben Takt ist erlaubt. Wann darf der Verbraucher EX beginnen? Bestimmen Sie die neue Fensterlänge und das CPI bei sonst unverändertem M1. (2 P)
3. Zwei Modellübersetzungen derselben Aufgabe benötigen hundert Befehle bei CPI eins beziehungsweise hundertfünfzig bei CPI 0,8. Die Frequenz ist konstant 100 MHz. Bestimmen Sie beide Zeiten. Ist das höhere IPC automatisch besser? (2 P)

Ihre Zeitachse und Rechnungen:

Aufgabe 2: Wertfluss und physische Versionen (6 Punkte)

Gegeben ist eine RV32I-Lehrfolge, RISC-V-Basis mit 32-Bit-Integerbefehlen. `s1` zeigt auf ein gültiges ausgerichtetes Wort mit Wert zehn. `t2 = 1`, `t3 = 7`, `t4 = 8` und `t5 = 3`.

```
lw  t0, 0(s1)     # A: Laden
add t1, t0, t2    # B: Addition
sub t0, t4, t5    # C: Subtraktion
and t6, t0, t3    # D: bitweises Und
```

1. Nennen Sie alle RAW-Kanten, Read After Write, zwischen diesen Befehlen. (2 P)
2. Wo liegen WAR, Write After Read, und WAW, Write After Write? Warum sind das Namenskonflikte? (1 P)
3. Anfangsabbildung: `t0` auf P7, `t2` auf P2, `t3` auf P3, `t4` auf P4, `t5` auf P5 und `s1` auf P9. Freie neue Ziele werden in Reihenfolge P10, P11, P12, P13 vergeben. Erstellen Sie für A–D eine Tabelle mit physischen Quellen und Ziel. Welche Quelle behält B nach der Vorbereitung von C? (2 P)
4. Bestimmen Sie die endgültigen Werte `t0`, `t1` und `t6`. Warum liest B keine fünf, auch wenn C intern früher fertig wird? (1 P)

P mit Zahl bezeichnet einen physischen Registerplatz. Renaming liest passende Quellabbildungen zuerst und vergibt anschließend ein neues Ziel. Die Quellen werden für den Leser festgehalten.

Ihre Kanten und Versionstabelle:

Aufgabe 3: Gleiche Ressourcen, andere Startreihenfolge (6 Punkte)

M3 verwendet A–D aus Aufgabe 2. Alle vier sind vor Takt eins dekodiert und umbenannt. Das Fenster zählt ausschließlich vom ersten Issue bis zum letzten Retire.

- Höchstens ein Start je Takt, eine ALU, Arithmetic Logic Unit, und eine LSU, Load/Store Unit.
- A startet in eins; sein Ergebnis ist erst Ende Takt vier bereit.
- Die anderen Operationen sind Ende ihres Starttakts fertig.
- Nutzer starten frühestens im Folgetakt ihres Produzenten.
- Höchstens zwei Retires pro Takt, strikt in Programmordnung und frühestens im Takt nach der jeweiligen Fertigstellung.

In-Order wählt nur den nächsten Programmplatz. Out-of-Order darf einen jüngeren bereiten Kandidaten wählen.

1. Zeichnen Sie den In-Order-Plan für Start, Fertigstellung und Retire. (2 P)
2. Zeichnen Sie den Out-of-Order-Plan. Wie nutzt C den zweiten Takt, und wann kann D beginnen? (2 P)
3. Berechnen Sie Fensterlänge, IPC und CPI beider Pläne. Bestimmen Sie Zeitreduktion und Beschleunigungsfaktor. Hat Out-of-Order den Load-Miss beseitigt? (2 P)

Ihre beiden Taktpläne und Bilanz:

Aufgabe 4: Vorhersage vor dem Update (5 Punkte)

Ein 2-Bit-Prädiktor startet in `11`. `00` und `01` sagen NT, Not Taken, nicht genommen, vorher, `10` und `11` T, Taken, genommen. Nach dem tatsächlichen T wird um eins bis maximal `11` erhöht, nach NT um eins bis minimal `00` vermindert.

1. Verfolgen Sie T, T, NT, NT, T in einer Tabelle: alter Zustand, Vorhersage, tatsächliches Ergebnis, Treffer/Miss, neuer Zustand. Wie viele Treffer entstehen? (3 P)
2. Ein separates additives Zeitmodell hat Basis-CPI eins. 20 % der bestätigten Befehle sind Sprünge; 10 % dieser Sprünge sind fehlvorhergesagt, je Fehler fallen effektiv fünfzehn zusätzliche Takte an. Bestimmen Sie CPI-Aufschlag, Gesamt-CPI, Zeitänderung und Durchsatzänderung bei gleicher Arbeit und Frequenz. Überlappende Strafen sind ausgeschlossen. (2 P)

Ihre Zustandstabelle und Prozentrechnung:

Aufgabe 5: Bestätigung und bleibende Spuren (5 Punkte)

Betrachten Sie zwei getrennte Varianten von M3. Vor A gelten `t0 = 42`, `t1 = 9`, `t6 = 12`. Ein Vorgänger hat `s0 = 13` bereits erfolgreich bestätigt. Ein ROB, Reorder Buffer, verwaltet Ordnung und Fertigstatus; Zahlenwerte liegen in der PRF, Physical Register File.

1. In der Fehlervariante meldet A statt eines Ergebnisses einen synchronen Load-Zugriffsfehler. C und D haben intern bereits fünf berechnet. Welche der genannten Registerwerte bleiben architektonisch erhalten? Welche Befehle dürfen keine normale Wirkung bestätigen? (2 P)
2. In der Erfolgsvariante liefert A zehn. Ein Interrupt wird an der ausdrücklich erlaubten Grenze nach A, vor Bestätigung von B und C, angenommen. Welche vier genannten Registerwerte sind dann gültig? Ist der Interrupt ein fehlerhafter Load? (1 P)
3. Ein davon unabhängiger Spectre-v1-Fall hat einen falsch vorhergesagten Bereichspfad. Der Pfad wird verworfen. Warum kann dennoch ein Cache-Seitenkanal entstehen? Welche Kopplung braucht der geladene Wert, und was beweist eine einzelne kurze Zugriffszeit? (2 P)

Ihre beiden Zustände und Seitenkanalbegründung:

Aufgabe 6: Zwei Akkumulatoren und richtige Nachweise (4 Punkte)

Der folgende vollständige C-Rechenhelfer liest vorzeichenlose 32-Bit-Werte und summiert in 64 Bit. `a` zeigt auf mindestens `n` gültige Elemente. `size_t` ist der vorzeichenlose Typ für Objektgrößen und Indizes. Die Integer-Typen stehen in `<stdint.h>`, `size_t` in `<stddef.h>`.

```
#include <stddef.h>
#include <stdint.h>

uint64_t reduce_split(const uint32_t *a, size_t n) {
    uint64_t sum0 = 0, sum1 = 0;
    size_t pairs = n / 2;
    for (size_t j = 0; j < pairs; ++j) {
        sum0 += a[2 * j];
        sum1 += a[2 * j + 1];
    }
    if (n % 2 != 0)
        sum0 += a[n - 1];
    return sum0 + sum1;
}
```

1. Verfolgen Sie `a = {1,2,3,4,5}` und `n = 5`: Paare, Teilwerte nach der Schleife, Restwert und Rückgabe. (1 P)
2. Warum sind `2*j` und `2*j+1` im Schleifenkörper gültige Indizes? Was geschieht bei `n = 0`, auch für `a = NULL`? (1 P)
3. Warum kann diese Form mehr unabhängige Arbeit anbieten als eine einzige Summenkette? Genügt bloßes Ausrollen derselben Kette dafür? (1 P)
4. Was zeigt Ergebnisgleichheit am Host? Was zeigt eine Assemblerausgabe, und was garantieren beide zusammen noch nicht über die Laufzeit oder `-O3`? (1 P)

Ihre Teilwerte, Indexbegründung und Nachweisgrenzen:

Optional: vorhandenes C-Programm übersetzen und beobachten

Die Datei `labs/src/lab07_ilp.c` enthält beide vollständigen Funktionen und eine Prüfung mit bekannten Ergebnissen. Starten Sie vom Repository-Hauptordner:

```
gcc -std=c11 -O1 -Wall -Wextra -Werror \
  labs/src/lab07_ilp.c -o /tmp/lab07_O1
/tmp/lab07_O1

gcc -std=c11 -O3 -Wall -Wextra -Werror \
  labs/src/lab07_ilp.c -o /tmp/lab07_O3
/tmp/lab07_O3
```

Beide Läufe sollen ausgeben:

```
n=8: Kette=36, geteilt=36
n=5: Kette=15, geteilt=15
Alle Prueffaelle bestanden.
```

GCC heißt GNU Compiler Collection. `-std=c11` wählt die C-Sprachfassung, `-O1` und `-O3` Optimierungsstufen, `-Wall` und `-Wextra` Warnungen, `-Werror` deren Behandlung als Fehler. `-o` benennt die ausführbare Datei. Die Shellfortsetzung `\` verbindet die beiden Kommandozeilen. Ein Compilerfehler muss behoben werden, bevor ein Lauf bewertet wird.

Die Datei prüft leere Eingaben, alle Präfixlängen null bis acht, gerade und ungerade Fälle sowie Werte bis `UINT32_MAX`. `reduce_chain` führt eine `uint64_t`-Summe fort; `reduce_split` führt zwei solche Summen und die Restbehandlung aus. `uint32_t` und `uint64_t` sind vorzeichenlose 32- und 64-Bit-Typen. `UINT32_MAX` ist der größte 32-Bit-Wert, `UINT64_C` erzeugt eine passend breite Konstante. `PRIu64` aus `<inttypes.h>` ist das passende Ausgabeformat, `%zu` das für `size_t`. `NULL` wird nur mit Länge null benutzt. Der Prozess meldet bei einem falschen Ergebnis einen Fehlerstatus; er misst keine Hardwarezyklen.

Zusätzlich können Sie im Compiler Explorer dieselben Funktionen für einen verfügbaren RISC-V-GCC betrachten. RISC heißt Reduced Instruction Set Computer. Wählen und dokumentieren Sie Compilerfassung, Zielarchitektur, ABI, Application Binary Interface, und Optionen. Beispielsweise sind `-march=rv32i` und `-mabi=ilp32` nur für einen passenden RISC-V-Compiler gültig; sie gehören nicht an den Host-GCC-Aufruf oben. RV32I ist die 32-Bit-Integerbasis, ILP32 eine 32-Bit-Schnittstellenkonvention.

Vergleichen Sie die **tatsächlich** ausgegebenen Befehle und notieren Sie, ob Ausrollen oder getrennte Teilketten sichtbar sind. `-O3` garantiert keines von beidem. Eine Renode-Ausführung ohne passendes Mikroarchitekturmodell oder ein skalares Ripes-Modell belegt keine Desktop-Out-of-Order-Zeit. Kein zusätzlicher Simulator ist für die Pflichtaufgaben nötig.